PatchProbe

Project study and speaking guide

Natural Language Processing | Team 8 | Navrachana University

Use this guide to learn the MVP and rehearse the 13-slide presentation. Scripts are written in simple spoken language and follow the current PowerPoint order.

The project in 30 seconds

PatchProbe reads one official Django release-note entry and suggests the first area a developer should verify after an upgrade. The five possible areas are API/code, configuration/deployment, data/schema, UI/output and no targeted check stated. A person makes the final call. The current version is a local research pilot, not a safety guarantee.

Who presents what

Slides	Presenter	Main topic
1-3	Bhargav Panchal	Title, problem and related work
4-6	Kavya Jariwala	Dataset, method and stack
7-9, 11	Tilaksinh Chauhan	Code walkthrough and live demo
10, 12-13	Om Nathani	Results, limits and close

Important numbers

14 pinned source files | 571 extracted entries | 50 human-confirmed reviews | 49 usable labels | 30 train / 9 dev / 10 test | 0.60 test accuracy | 0.276 five-label macro-F1 | 23 passing tests

Read the weak result honestly: the keyword baseline has macro-F1 0.280, slightly above the trained model. One target class has no examples.

How the MVP works

Official Django RST files -> meaningful note entries -> human-confirmed labels -> split by whole release -> word and character TF-IDF -> balanced logistic regression -> app suggestion and error review.

What goes into the model

Only cleaned note text. Release version, source URL, section heading and raw RST stay in the dataset for traceability, but they are not model inputs.

What the app can and cannot do

- Can: show the original note and source link, save a reviewed label, resume a queue, and suggest an area for pasted text.

- Cannot: know whether a particular app uses the changed feature, run upgrade tests, or promise that an upgrade is safe.

Five labels in one line each

Label	First check
API / code	Calls, imports, signatures, hooks and integration code
Configuration / deployment	Settings, runtime, packaging or environment
Data / schema	Database schema, migrations or stored data behavior
UI / output	Rendered screens, forms, responses or user-visible output
No targeted check stated	The note supports no concrete downstream check

UNCLEAR is separate: use it when the note fairly points to more than one first check or is too vague. It is not a model label.

Slide-by-slide speaking scripts

Read naturally, not word for word. Keep the PowerPoint visible and use the prompt below each script for where to point or what to open.

Slide 01 | PatchProbe: title and introductions

Presenter: Bhargav Panchal | **On screen:** Title, task sentence, names and IDs.

Say: Hello. We are Team 8 from Navrachana University. Our project is PatchProbe. It reads a software release-note entry and suggests the first area a developer should verify after an upgrade. We built a working local pilot using Django release notes. We will show the problem, how the data and model work, a live app, the actual pilot results, and what we need to improve next.

Cue: Continue to slide 2 with the SQLite example.

Slide 02 | The upgrade review problem

Presenter: Bhargav Panchal | **On screen:** SQLite example; current three-step review; MVP audience.

Say: A release note tells us what changed, but it does not give a test plan for our application. Here the minimum SQLite version rises from 3.27.0 to 3.31.0. A developer still has to check whether the app uses SQLite, where it runs, and whether the upgrade affects it. Today, a person reads notes, maps each change to their app, then chooses what to check first. PatchProbe helps with that last sorting step. For the SQLite note, it suggests configuration and deployment. It does not know the app's actual setup or guarantee that the upgrade is safe. Developers, QA reviewers and maintainers are the users.

Cue: Point at the SQLite example, then move to the comparison on slide 3.

Slide 03 | Related work and task distinction

Presenter: Bhargav Panchal | **On screen:** Table: DeepRelease, Wu et al., Pan et al., PatchProbe.

Say: We checked three nearby research directions. DeepRelease works earlier in the process: it uses pull requests to create release notes and classify change types. Wu and colleagues study issues about release notes. Pan and colleagues survey machine-learning methods for choosing tests. Our input is a published release-note entry. Our output is the first broad area a developer should verify: for example, code, configuration, data or user-visible output. That exact task and the human review flow are our scoped contribution. We are not saying no one in the world has done something similar. We are saying the papers we reviewed focus on different inputs or outputs.

Cue: Use the Input and Output columns to explain the difference; hand over to Kavya.

Slide 04 | Dataset, labels and pilot coverage

Presenter: Kavya Jariwala | **On screen:** 14 files; 571 entries; 50 human-confirmed; 49 usable; class chart.

Say: We used official Django release notes. We pinned 14 source files and extracted 571 candidate entries. One row is a meaningful note, usually a bullet or short paragraph. Each row keeps its release, source link, revision, raw RST text and cleaned text. The extracted rows are not automatically labeled. We have a separate pilot of 50 human-confirmed decisions, with ten entries from each of five Django releases. One row is UNCLEAR, so 49 are usable for model training. The chart shows 29 API or code, 9 UI or output, 6 configuration, 5 data or schema, and zero no-targeted-check examples. That imbalance matters later.

Cue: Do not call UNCLEAR a sixth model class. It is a review state.

Slide 05 | End-to-end NLP methodology

Presenter: Kavya Jariwala | **On screen:** Six stages and four controls.

Say: Our pipeline has six steps. First, we use pinned RST release-note files. Second, we extract clean entries and keep IDs, URLs and source hashes. Third, people review a 50-row pilot with a written label guide. Fourth, we split whole releases into train, development and test groups. Fifth, the model turns words and character pieces into TF-IDF features and uses logistic regression to choose an area. Sixth, the app shows a suggestion and we review errors. The model sees only cleaned note text. The vectorizers learn from training releases only. Exact duplicates across splits stop training, and similar pairs are flagged. We compare with simple baselines and report macro-F1 and class errors.

Cue: Explain that there is no stopword-removal stage or fixed embedding size.

Slide 06 | Technology stack and design choices

Presenter: Kavya Jariwala | **On screen:** Python; scikit-learn and NumPy; Streamlit; joblib; pytest and Git.

Say: The stack is small enough to run on a laptop. Python handles source files, text cleaning, CSV output and hashes. Scikit-learn builds TF-IDF features, trains logistic regression and computes evaluation scores. NumPy supports the numeric work. Streamlit gives us the review and prediction screens without a separate frontend. Joblib saves the trained local pipeline so the app can load it. Pytest checks the important behavior, and Git keeps the work reproducible. We have 23 passing tests in this build. The committed offline demo needs no paid API or credentials. Tilaksinh will now show where these parts live in VS Code.

Cue: Open pyproject.toml if asked which packages are declared.

Slide 07 | VS Code walkthrough: repository map

Presenter: Tilaksinh Chauhan | **On screen:** app.py; src/patchprobe; scripts; data/processed.

Say: This is the real project folder in VS Code. app.py is the Streamlit interface. The src/patchprobe folder holds the core code: ingest.py reads and cleans notes, annotation.py checks and saves human decisions, model.py trains and evaluates, and inference.py loads a model for predictions. The scripts folder gives us commands to build the dataset, train the model and check near duplicates. The processed data folder has the extracted corpus and the shareable pilot export. Follow one entry: build_dataset.py creates it, annotation.py records its reviewed label, train_model.py uses it, and inference.py can suggest a label for new text.

Cue: Expand the Explorer first, then click the files in this order.

Slide 08 | VS Code walkthrough: extraction and review

Presenter: Tilaksinh Chauhan | **On screen:** Abridged build loop; annotation checks and atomic save.

Say: This slide shows short versions of the real code. The build script loops through pinned source groups and releases. It fetches the exact source revision, parses entries, removes exact duplicates and selects the 50-row pilot. In VS Code I can show SOURCE_GROUPS and the entry fields in ingest.py. Annotation code then checks that the entry ID exists. If a reviewer chooses UNCLEAR, it requires a reason. The chosen label is saved with the entry ID and reviewer information. The program writes a temporary file and replaces the old CSV in one step, so a half-written file is less likely. We keep the original source link and cleaned model text for every entry.

Cue: Say that the slide snippets are shortened; the live source files are exact.

Slide 09 | VS Code walkthrough: training and inference

Presenter: Tilaksinh Chauhan | **On screen:** Word and character TF-IDF; balanced logistic regression; three guards.

Say: In model.py, the pipeline joins two kinds of TF-IDF features. Word one- and two-word groups capture terms like 'database setting'. Character groups of length three to five help with names, code identifiers and spelling changes. Logistic regression then predicts a class. We use balanced class weights because the pilot labels are uneven. The split is by whole release, and the TF-IDF vocabulary is fitted only on training rows. For this pilot, the sparse vocabulary has 5,838 features; we did not choose a dense embedding size. In inference.py, the app loads a local model only when its matching metrics file is present. A class missing from training is shown as 'Not trained in this pilot.'

Cue: Show split_by_release, make_pipeline and predict_review in the live code.

Slide 10 | Pilot evaluation and baseline comparison

Presenter: Om Nathani | **On screen:** 30 train, 9 dev, 10 test; accuracy and macro-F1 chart.

Say: We tested on ten notes from Django 6.0, which was held out from training. The split has 30 training rows, 9 development rows and 10 test rows. Test accuracy is 0.60, which means six of ten predictions matched the reviewed labels. Five-label macro-F1 is 0.276. The majority baseline has macro-F1 0.133. A fixed keyword baseline scores 0.280, just above our model. The model predicted API or code for nine of ten test rows, and one target class has no pilot examples. So the pipeline works, but these results do not show a reliable five-class product. We show the weak result because it tells us what to improve.

Cue: Never call 0.276 accuracy. Do not claim the model beats the keyword baseline.

Slide 11 | Live MVP demonstration

Presenter: Tilaksinh Chauhan | **On screen:** Start command; guided annotation; prediction review.

Say: Now I will show the running app. In Guided annotation, the pilot queue shows 50 out of 50 reviewed. I will revisit a saved note so you can see the text, source link and chosen label. Then I will open Prediction review and paste the SQLite sentence from slide 2. The current pilot model suggests Configuration and deployment. The screen also shows a warning that this is a small pilot. The fifth class has no training examples, so the app marks it as not trained. This output is a starting point for human review, not a promise that an upgrade will work. If this screen is missing on a fresh laptop, the local model must be trained first.

Cue: Prepare the terminal and app before class; use the exact demo runbook.

Slide 12 | Current limits and roadmap

Presenter: Om Nathani | **On screen:** Class imbalance; tiny test; near duplicate; independent review; future scope.

Say: The pilot tells us the next steps clearly. Twenty-nine of 49 usable rows are API or code. One target class has zero examples. The test set is only ten entries from one Django release. We also found one very similar deprecation and removal pair across development and test, so that result needs care. Next, our four teammates will each independently re-review about 12 or 13 pilot rows. We will collect more examples of rare and no-targeted-check notes, set a clear rule for near-duplicate families, and test on a larger frozen split. Only after stronger results should we try other libraries, repository integration or live APIs.

Cue: The 12-13-row per-person split is future work, not a claim about the first pilot.

Slide 13 | Final deliverable and conclusion

Presenter: Om Nathani | **On screen:** Six ready deliverables and next milestone.

Say: Our deliverable today is a working, documented review pilot. We have 14 pinned Django source files and 571 extracted entries. We have 50 human-confirmed pilot decisions, with 49 usable for the model. The local Streamlit app supports review and prediction. The training pipeline splits by release and uses TF-IDF with logistic regression. We also provide baseline comparisons, split records, error outputs, tests and source links. The code and shareable pilot export are in the project repository. The trained model remains a local artifact. The next milestone is a larger dataset with independent review and stronger per-class evaluation. Thank you; we are ready for questions.

Cue: Close with the repository and let the group answer questions.

Live demo: fast rehearsal card

Before class

- Open the PatchProbe project root in VS Code. Keep files from slides 7-9 visible in the Explorer.
- From the project root, run the demo checks in docs/DEMO_RUNBOOK.md. Train the local model if Prediction review is not available.
- Start Streamlit with: `.\.venv\Scripts\python.exe -m streamlit run app.py`
- Check that Guided annotation reads 50 / 50 on the original laptop. A fresh clone may need the committed pilot export copied into the local annotation path.

During the demo

Step	Show	Say
1	Guided annotation	These are saved human-confirmed pilot decisions; we can revisit a row.
2	Prediction review	Paste: The minimum supported version of SQLite is increased to 3.31.0.
3	Suggested area	Current pilot suggests Configuration/deployment; a person still checks the app.
4	Pilot warning	One class was not trained, and scores are not calibrated.

If asked to show code

Open pyproject.toml, then ingest.py and build_dataset.py, annotation.py, model.py, and inference.py. The snippets on the slides are shortened; the VS Code files are the real implementation.

Glossary

Release note: A published description of a change in a software version.

RST: reStructuredText, the source format used by the Django release-note files.

Entry / row: One meaningful release-note bullet or short paragraph used as a single input.

Label: The first area a developer should verify: API/code, configuration/deployment, data/schema, UI/output, or no targeted check.

UNCLEAR: A separate human review state when one first area cannot be chosen fairly. It is excluded from training.

Token / n-gram: A piece of text; an n-gram is a sequence of words or characters. The model uses word 1-2 and character 3-5 grams.

TF-IDF: A way to turn text into numeric features. A term gets more weight when it is useful within the training text.

Sparse features: A long feature vector where most values are zero. The pilot has 5,838 learned features.

Logistic regression: A simple classifier that uses the text features to score the observed classes.

Class weight: A training setting that gives rare classes more weight. It cannot invent examples for a missing class.

Train / dev / test: Train fits the model; dev helps inspect development; test is held out for the reported result.

Release-group split: All labeled notes from one Django release stay in the same split.

Leakage: Information from held-out data getting into model fitting or evaluation, making scores misleading.

Baseline: A simple method used as a comparison, such as majority class or fixed keywords.

Accuracy: The share of test rows with the correct label; here 6 of 10, or 0.60.

Macro-F1: The average F1 across target classes, giving each class equal weight. Here five-label macro-F1 is 0.276.

Provenance: The source link, release and revision that let us trace an entry back to official text.

joblib: The file format used to save and load the local trained scikit-learn pipeline.

Likely professor questions

Give a short direct answer first. Add a detail only if the professor asks for it.

What problem does PatchProbe solve?

It suggests the first verification area for a published dependency release note. It helps route human review; it does not test an app.

What is novel here?

The specific input and output: a published note entry goes to a broad first-check area, supported by a label guide and reviewer workflow. Related papers solve different nearby tasks. We do not claim a worldwide first.

Why Django?

Its official release notes are public, structured and available at pinned source revisions. This keeps the MVP reproducible. We have not shown transfer to other libraries.

How many classes?

Five target classes. The current usable pilot contains four of them. UNCLEAR is a separate review state, not a sixth model class.

Why only 49 training labels from 50 reviews?

One reviewed note was marked UNCLEAR and is excluded instead of forcing a class.

How do you know the labels are correct?

They are human-confirmed under a written guide, but we have not yet measured independent reviewer agreement. A second pass is the next step.

Why no stopword removal?

Small words and negation can change technical meaning. TF-IDF handles common terms, so we keep the cleaned wording.

Why not a transformer or LLM?

The labeled pilot is very small. A CPU-friendly TF-IDF and logistic-regression baseline is faster to inspect and repeat. We need more reliable labels and evaluation before comparing larger models.

Why word and character features?

Words capture terms and phrases. Character pieces can capture identifiers and spelling changes common in code-related notes.

Why split by release instead of random rows?

Notes from one release can share wording and context. Holding out whole releases lowers one leakage risk.

Does the model beat simple methods?

It beats the majority baseline on macro-F1, 0.276 versus 0.133. It is slightly below the keyword baseline, 0.280. Ten test rows are too few for strong ranking claims.

What does 0.276 mean?

It is five-label macro-F1, not accuracy. Accuracy is 0.60. Macro-F1 gives each intended class equal weight and shows poor class coverage.

Can scores be treated as confidence?

No. The pilot scores are not calibrated. The app warns users and marks the class absent from training.

Will this catch every breaking change?

No. It suggests one broad first-check area from note text. A developer still reads the note, checks app usage and runs tests.

What would make it production-ready?

More independently reviewed examples across all classes, stronger held-out and cross-library tests, duplicate control, calibrated scores and a human escalation path.

Sources and final checks

Read these project files before presenting

- presentation/README.md - current speaker assignment and slide order.
- docs/DATA_AND_LABELS.md - label rules and UNCLEAR handling.
- docs/ARCHITECTURE.md - file map and pipeline.
- docs/MODEL_CARD.md and docs/EVALUATION.md - exact pilot design, scores and limits.
- docs/DEMO_RUNBOOK.md - commands and fallback steps.

Related work referenced on slide 3

DeepRelease (2022): https://arxiv.org/abs/2201.06720
Wu et al. (2022): https://arxiv.org/abs/2203.15592
Pan et al. (2021): https://arxiv.org/abs/2106.13891

Source repository and software notes

Project: https://github.com/TilakCSE/PatchProbe
Django upgrade guidance: https://docs.djangoproject.com/en/6.0/howto/upgrade-version/

Final three reminders

- Say 'human-confirmed pilot' for the 50 decisions. Do not assign past review counts to individual teammates.
- Say 0.60 accuracy and 0.276 macro-F1; do not swap these metrics.
- Describe the app as a working local MVP and the result as a small pilot.

Prepared for Team 8 | October 2026